

The traveling salesman problem

Content

1. The traveling salesman problem
2. Greedy algorithm
3. Monte-Carlo-algorithm
 1. Algorithm
 2. Analysis of parameter
4. Comparison
5. Optimization of the result
6. Further more complex Problems

(2)

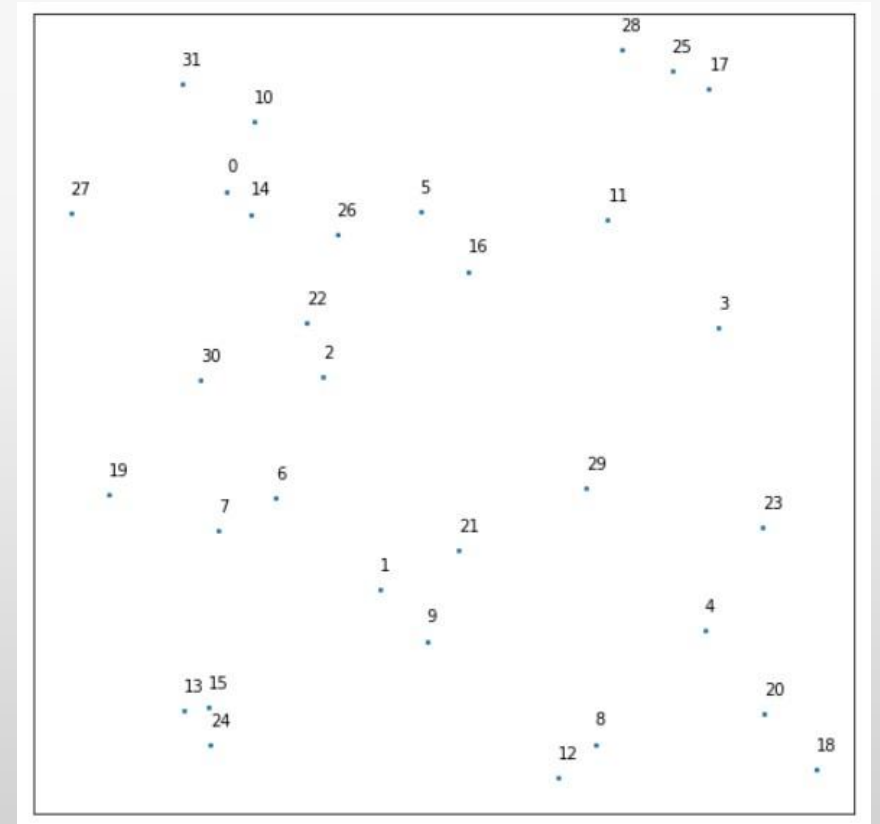
The traveling salesman problem

The traveling salesman problem

- A traveling salesman has to find the shortest closed tour between a set number of cities N
- All **distances** between the cities are known
- Problem: $(N-1)!$ possibilities
- Solution: use algorithm
 - Only quasi-optimal results

Our example

- Distribution of 32 randomly distributed points $\vec{x} = \begin{pmatrix} x \\ y \end{pmatrix}$ with $x, y \in [0,2]$
- $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$
- Symmetric matrix



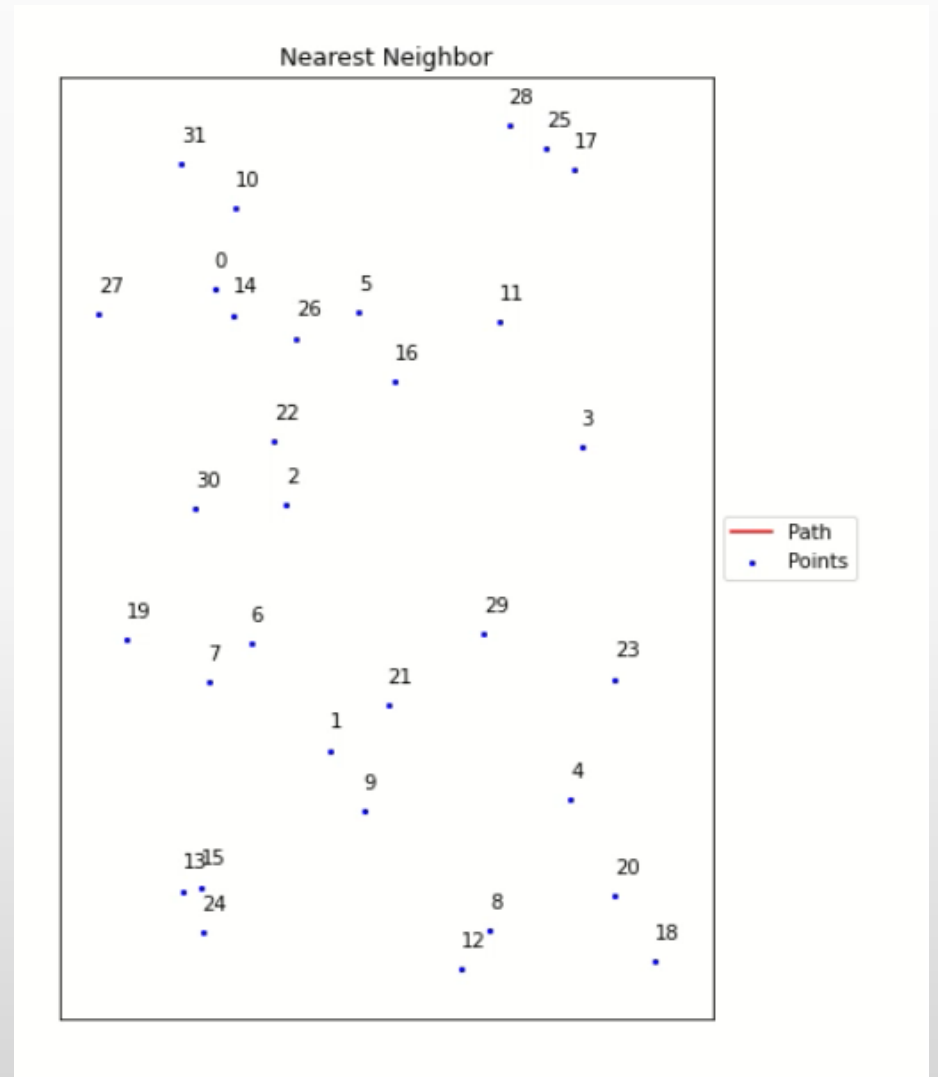
Greedy algorithm

Greedy Algorithm

- problem-solving heuristic of making the **locally** optimal choice at each stage
- Looking at three different approaches:
 - Nearest Neighbor
 - Double-sided nearest Neighbor
 - Nearest Insertion

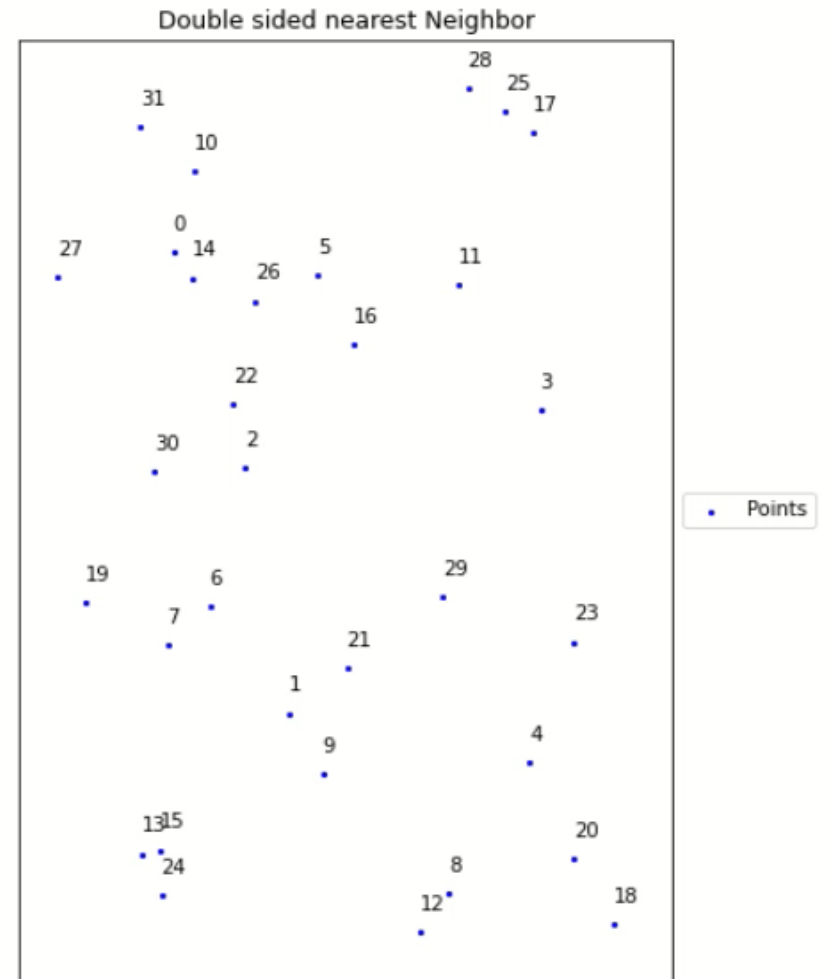
Nearest Neighbor [1]

- Start with a random start position
- Add always the cheapest edge to an unused vertex, until all vertices are visited once
- Add the starting vertex
- Do it for all start positions and choose the shortest path



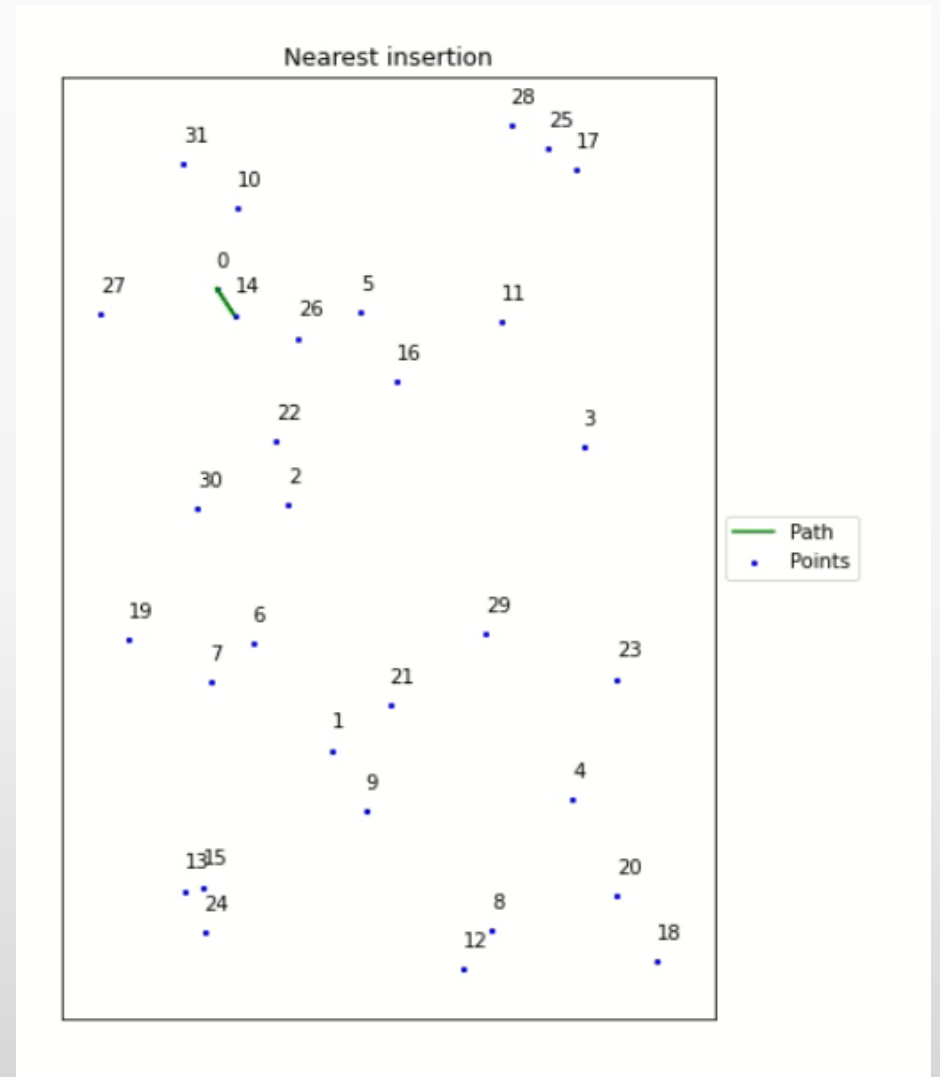
Double- sided Nearest Neighbor^[1]

- Same algorithm as for nearest Neighbor, but go both ways
- Start with the shortest edge in one direction and second shortest in the other



Nearest Insertion ^[2]

- Start with a circle
- Add vertex, so that the circumference of the circle stays the lowest for all possibilities



Monte-Carlo Markov-Chain

MCMC Algorithm _[3]

- Idea : Instead of creating a path using certain conditions, alter a previous path in a certain way.
- Code:
 - Start with random tour and calculate total distance D
 - For a number of steps
 - Change tour with optimization method and calculate D^*
 - If $x \sim Unif[0,1] < \exp\left(\frac{D-D^*}{T}\right)$ accept new tour else use old tour
 - If accepted $D = D^*$
 - $T_{new} = \alpha \cdot T$

Optimization Method_[3]

- Optimization method: randomly between 2-Opt and 3-Opt method
- 2-Opt method: remove 2 edges: reconnect subtour to opposite edge



- 3-Opt method: remove 3 edges: exchange connections between outer subtours

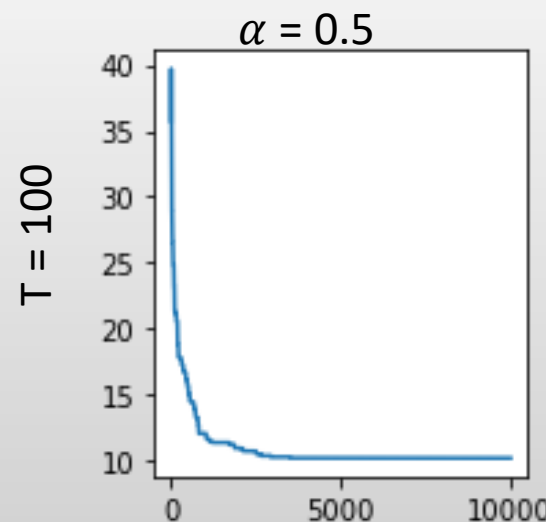
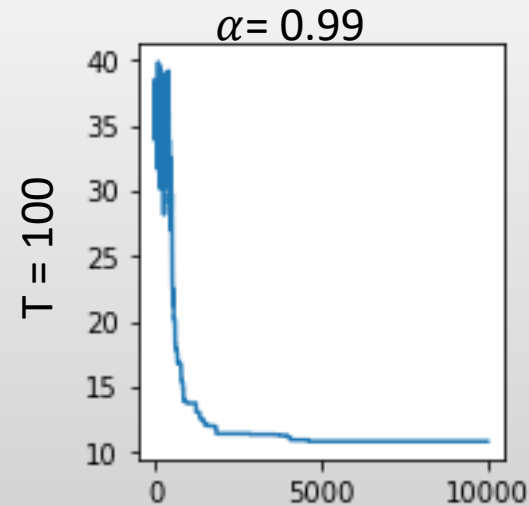
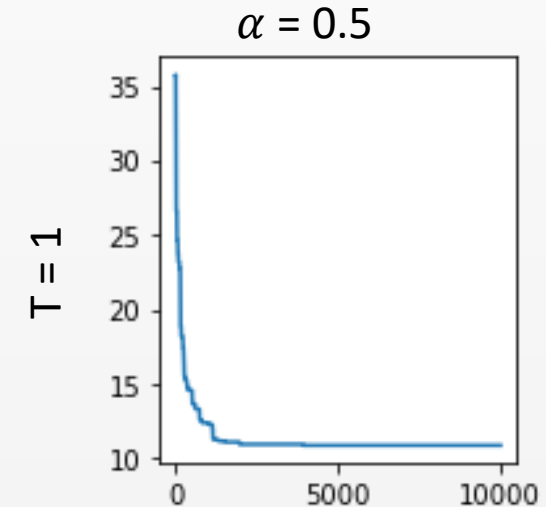
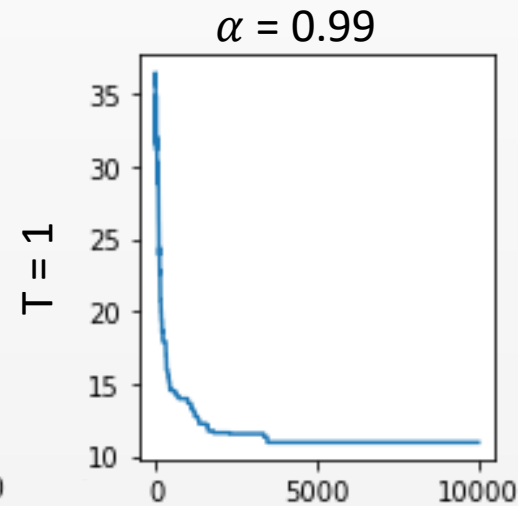
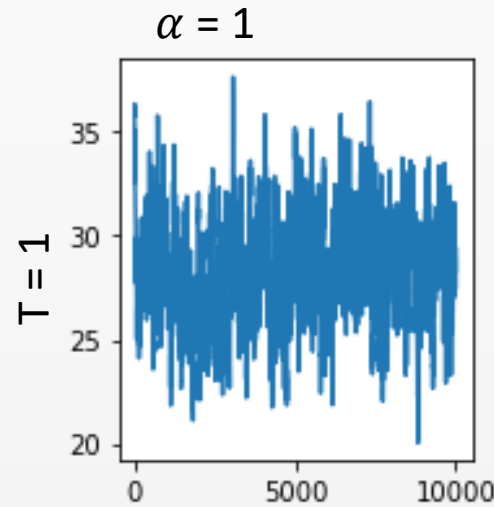


Role of T and α

- T and α serves as control parameter
- By lowering the T parameter, the system is transferred from a random high-cost to a feasible low-cost solution
- Often accept changes at the beginning , but later only accept cheaper paths
 - escape local minima at the beginning
 - trapped in a minima at the end -> might not be global Minima

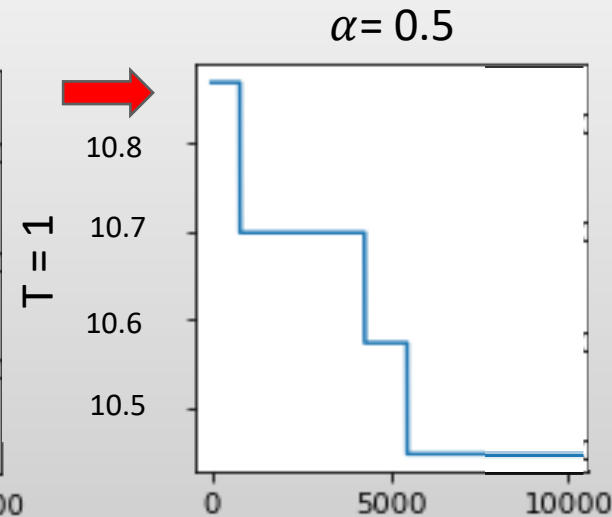
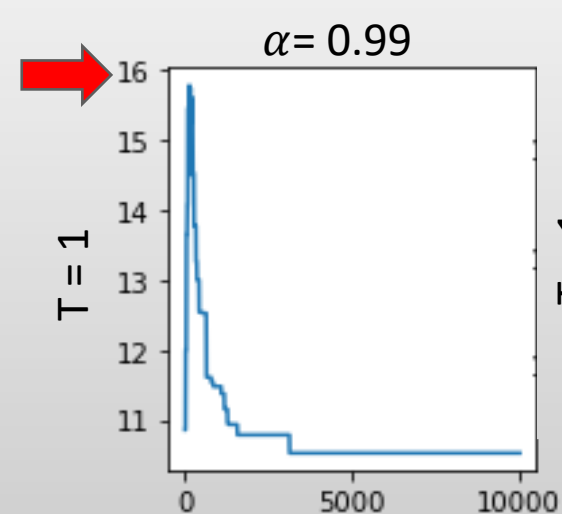
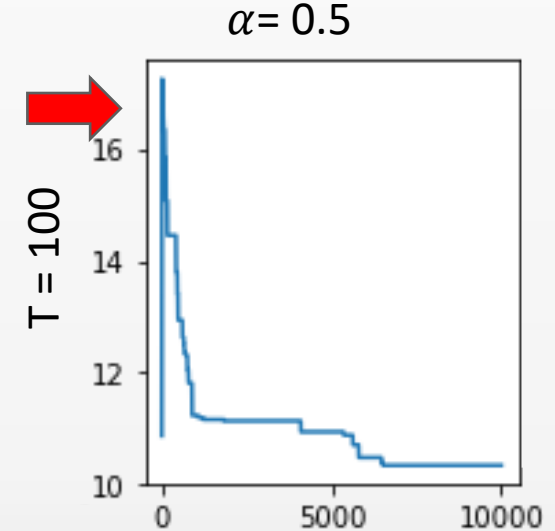
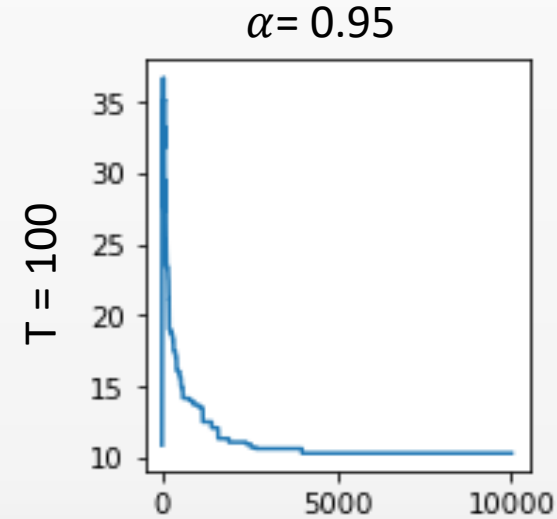
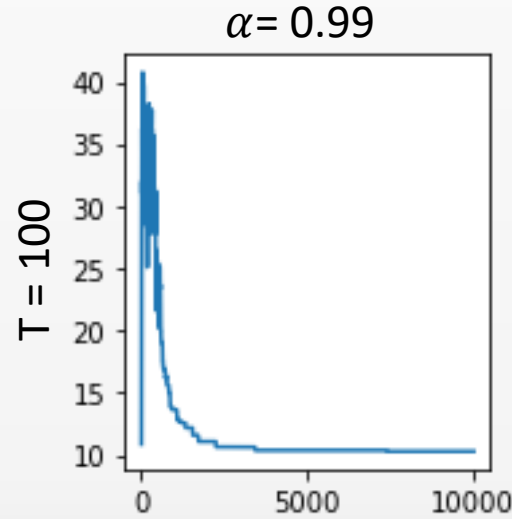
T and α - non optimal start

- Result for distance of different α and T with 10000 steps MCMC starting with 0,1,...,32
- We have $\exp\left(\frac{D-D^*}{T}\right)$ with $\max(D - D^*) \approx 2,5$

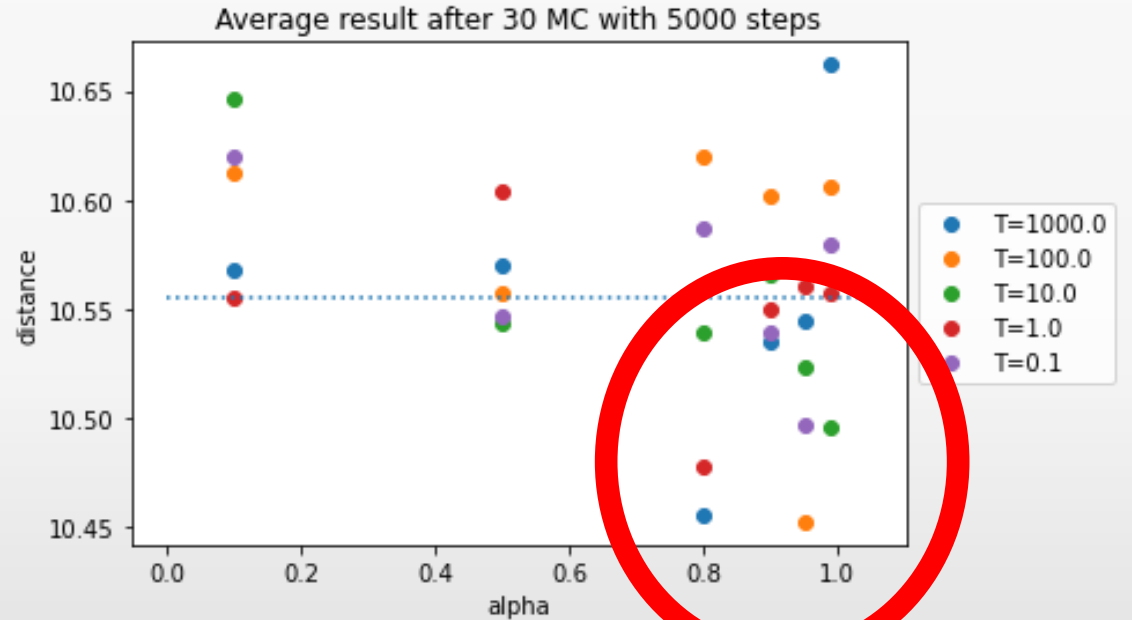
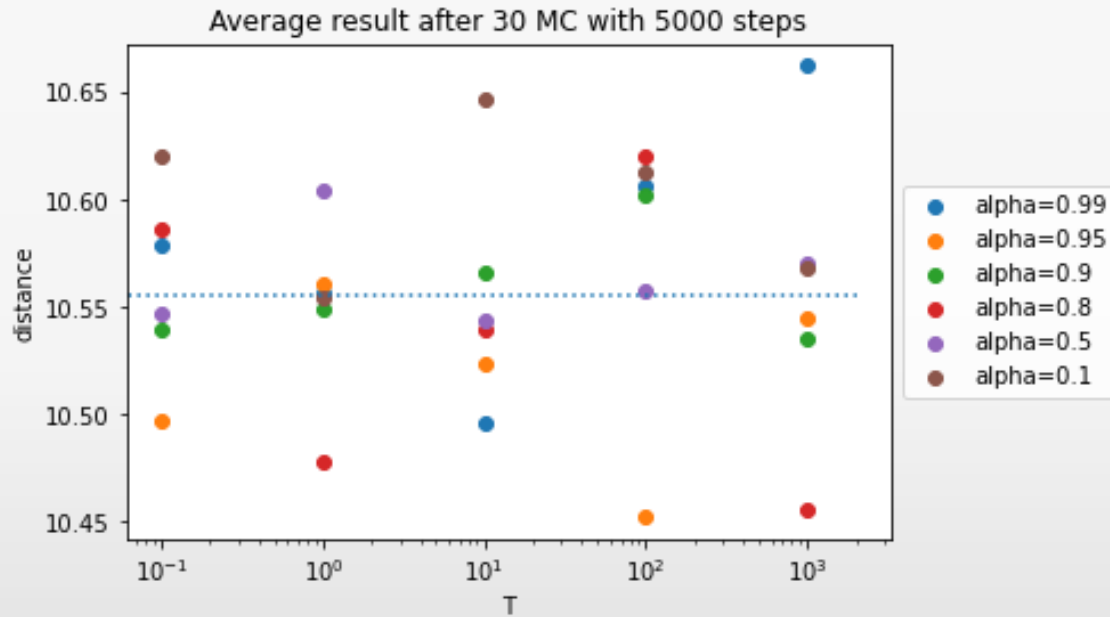


T and α – start in local minimum

- Result for distance of different α and T with 10000 steps MCMC starting in local minimum (result of previous MCMC)

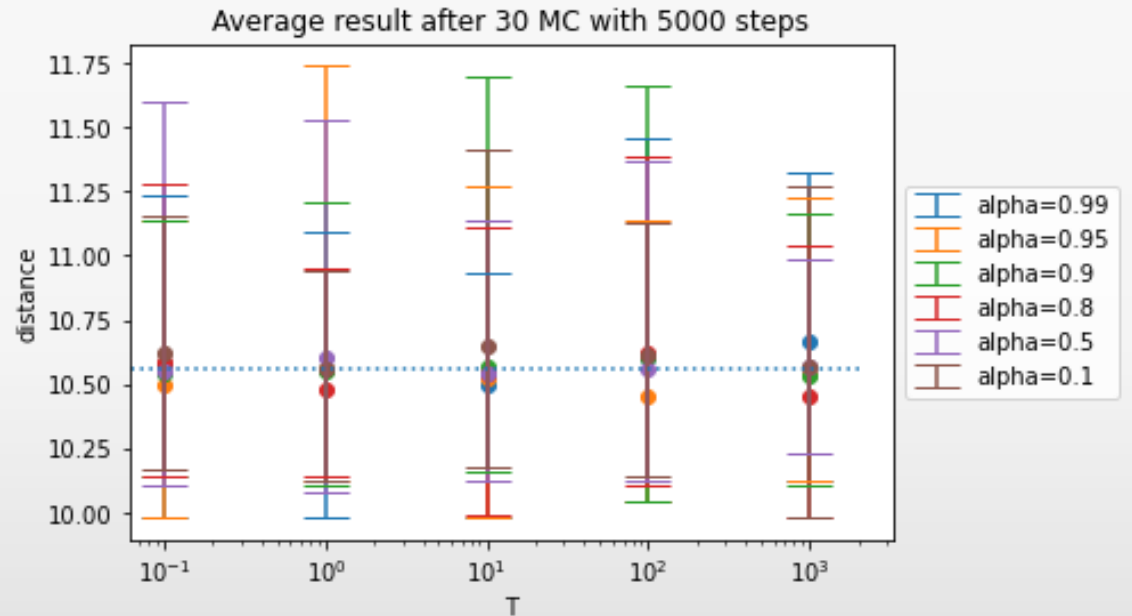
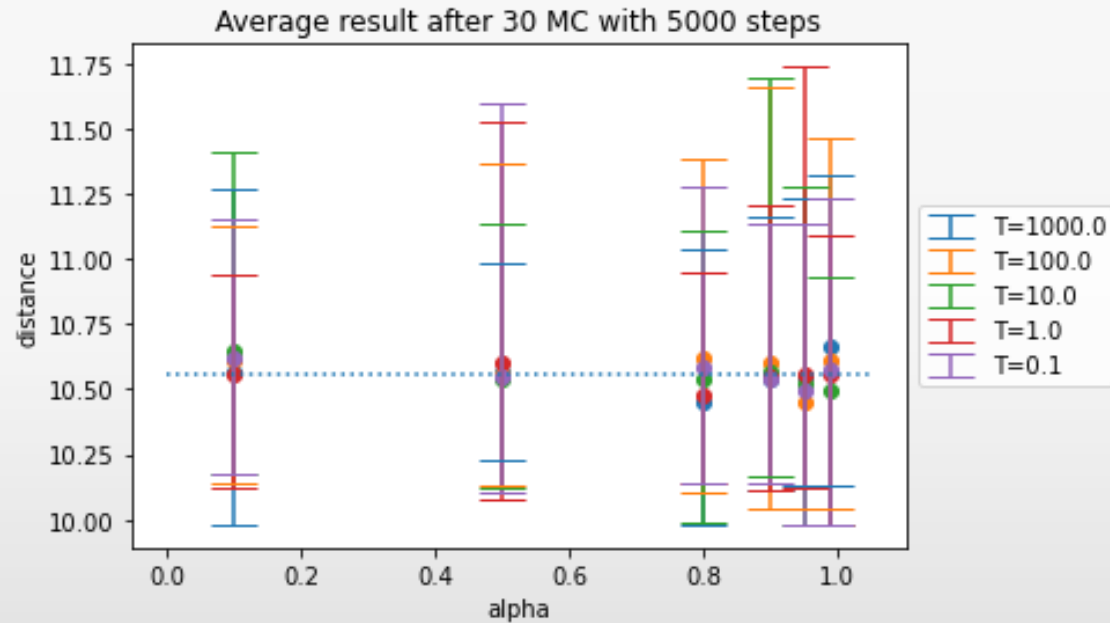


T and α averages over 30 MCMC



- In Average lower result for $\alpha > 0.8$

T and α spread of result over 30 MCMC



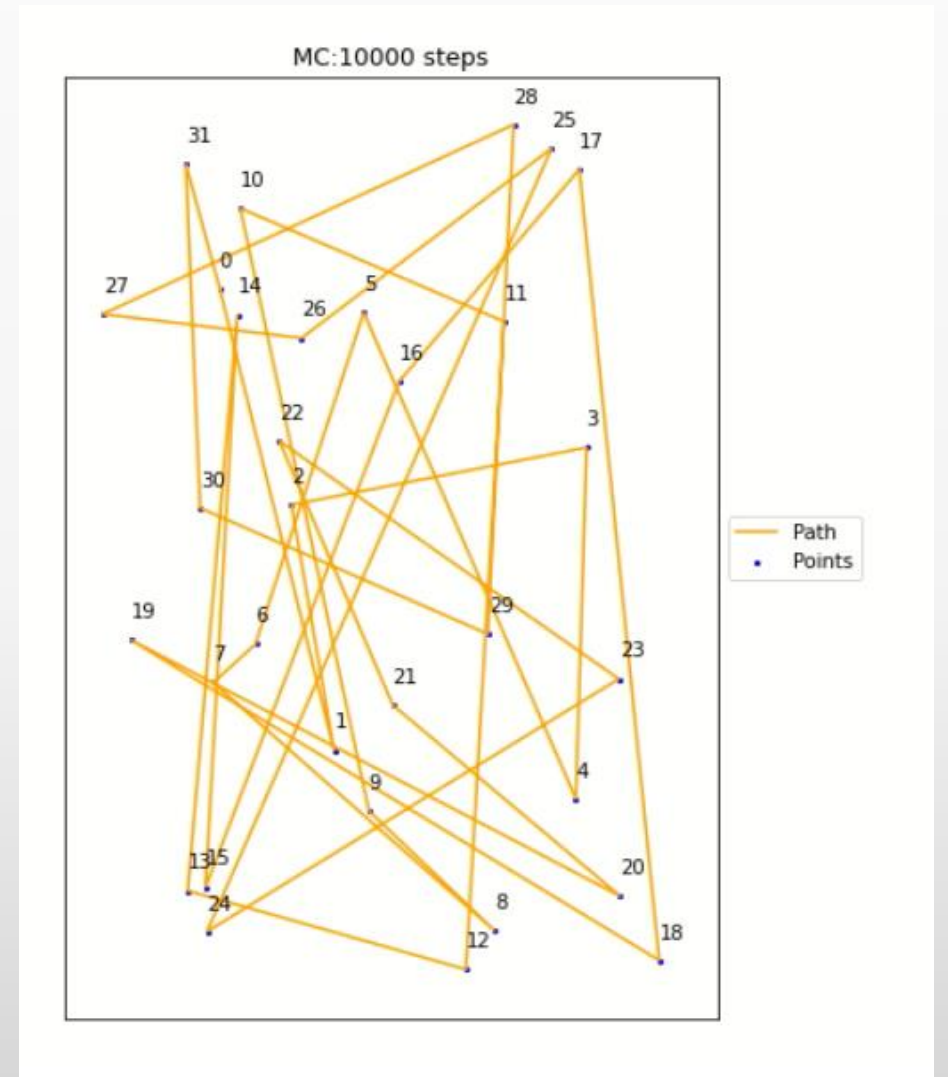
- Repeated MCMC leads to good result independently of chosen α and T due to randomness

T and α - Summarize

- High $T \sim 10 \cdot \max(D)$
- $0,8 \leq \alpha < 1$
- For our example:
 - Use 10000 steps
 - $T=100$ and $\alpha=0,95$

Monte-Carlo - Animation

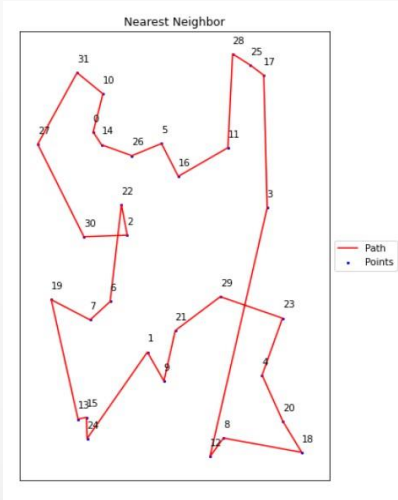
- Start with sequence 0,1,...,32
- 20 frames per animation step



Comparison

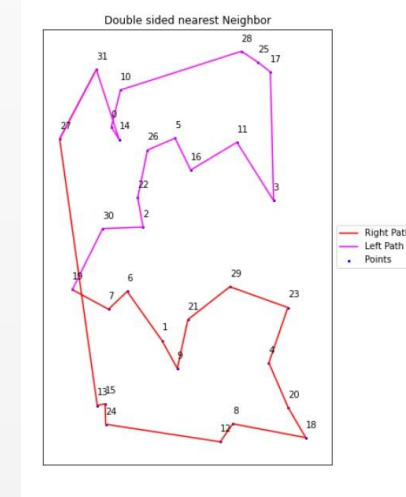
Comparison - Time

Nearest Neighbor



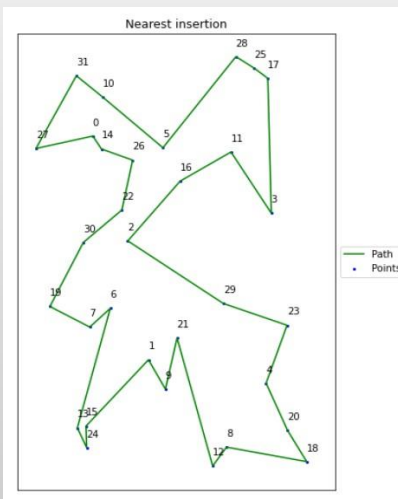
Time:
 $68.2 \text{ ms} \pm 2.63 \text{ ms}$

Double- sided nearest Neighbor



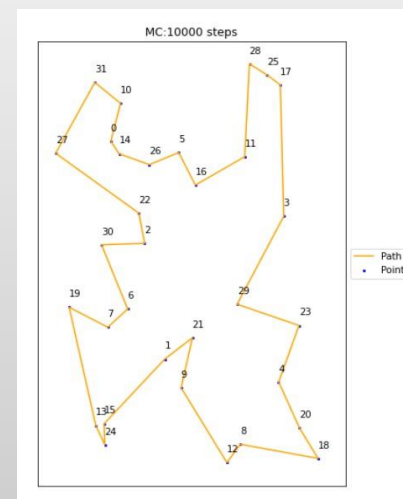
Time:
 $77.5 \text{ ms} \pm 5.04 \text{ ms}$

Nearest Insertion



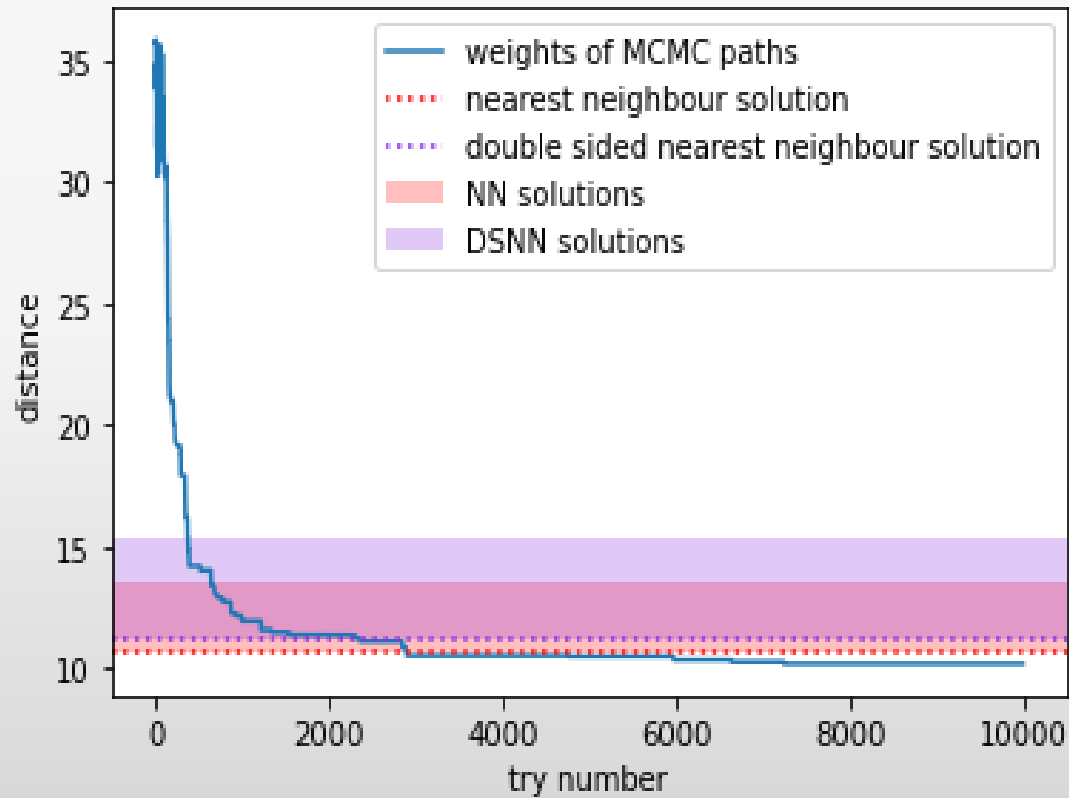
Time:
 $62 \text{ ms} \pm 1.11 \text{ ms}$

MCMC

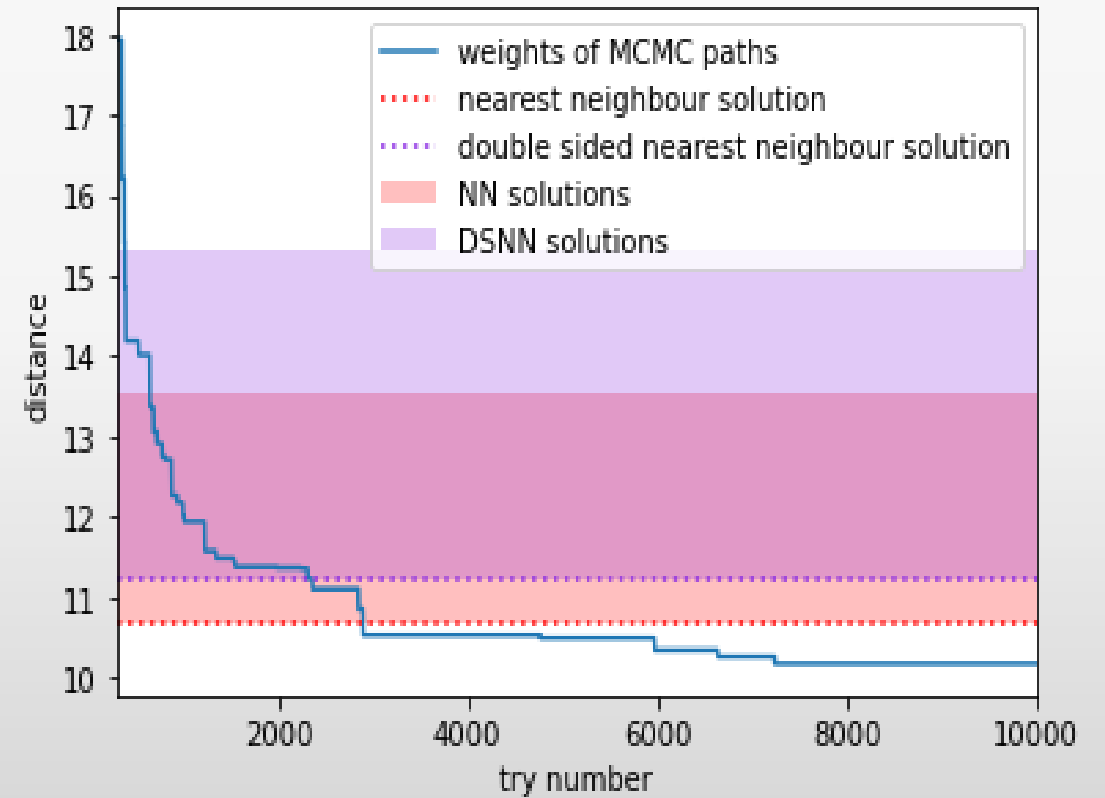


Time:
 $1.16 \text{ s} \pm 140 \text{ ms}$

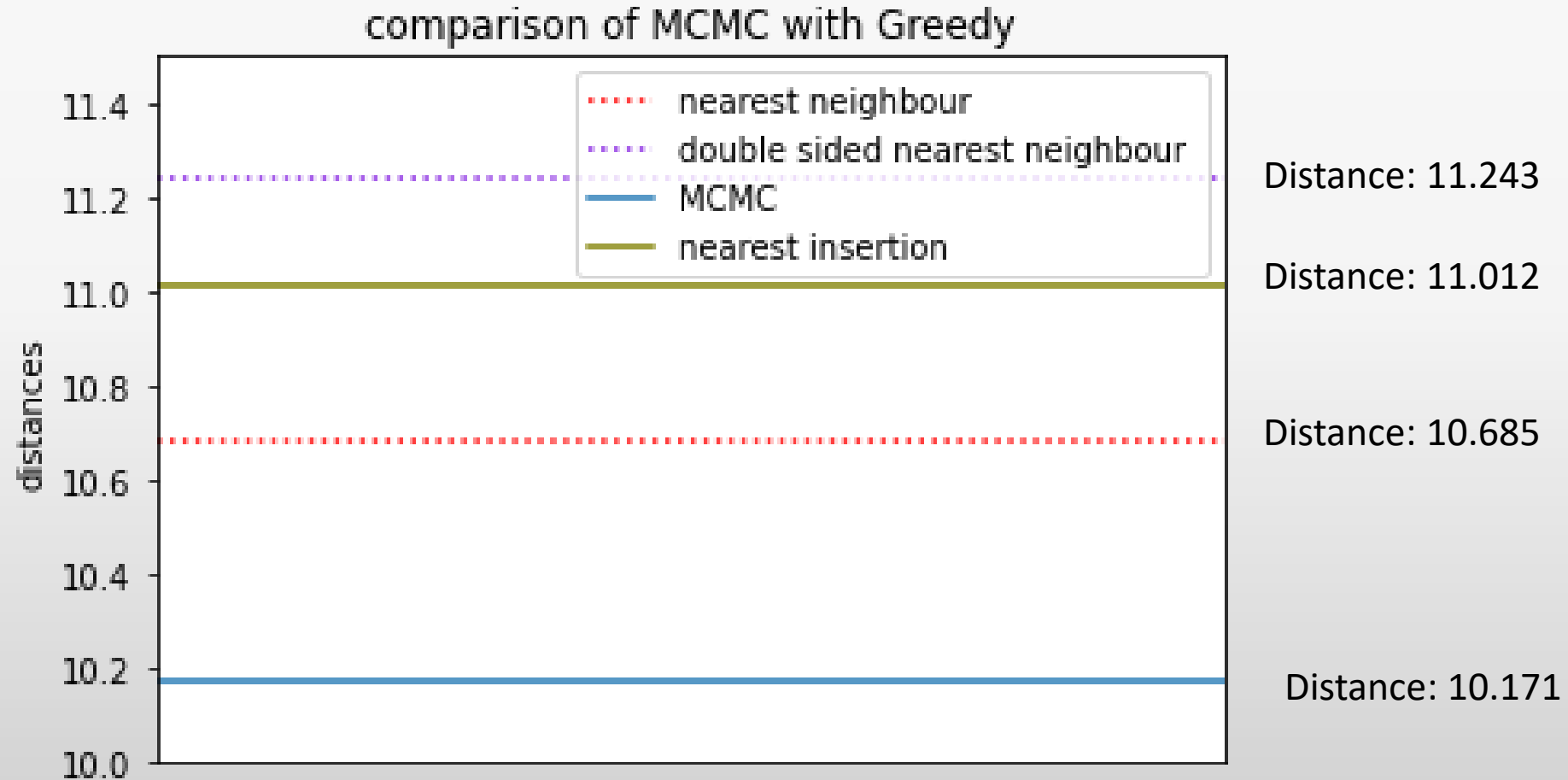
Comparison – Distances



After 300 steps



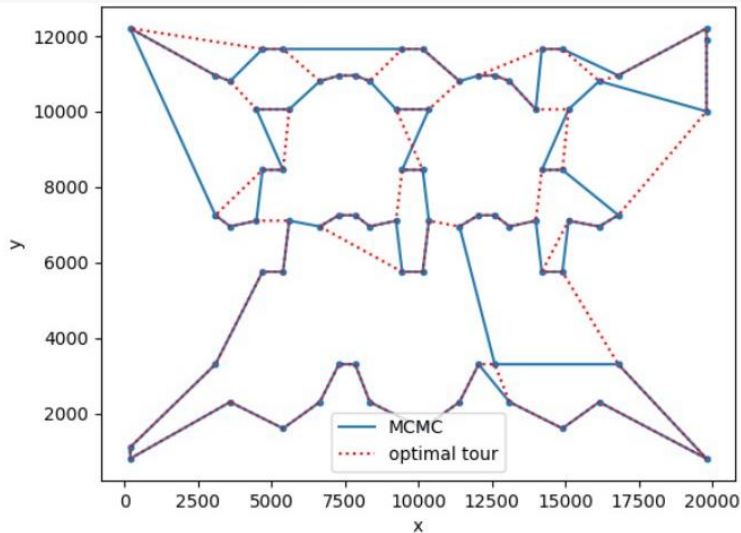
Comparison - Distances



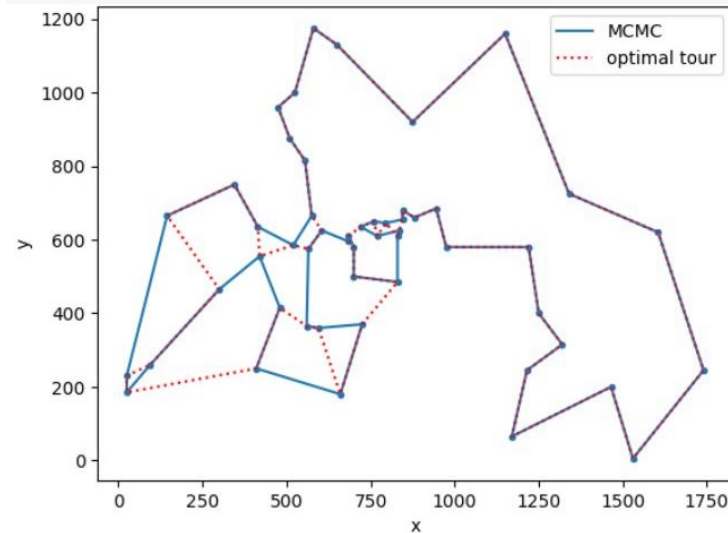
Comparison with known optimal solution

Optimal result from: <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/>

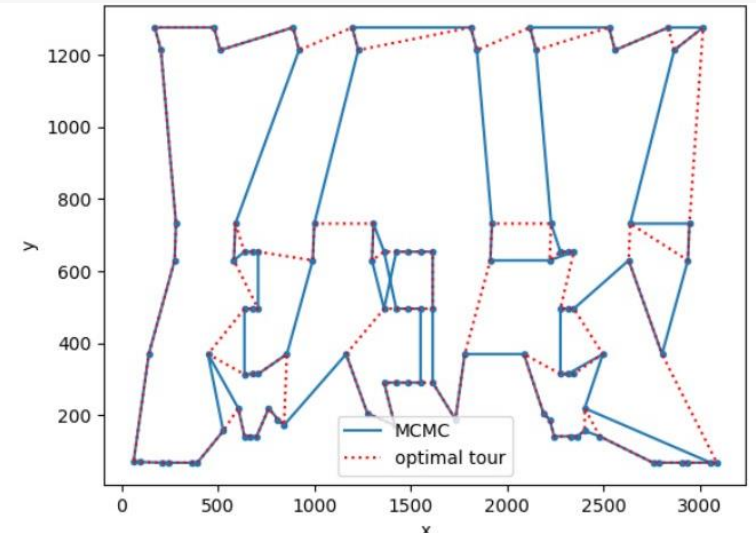
pr76



berlin52



lin105



Difference to optimal tour in %		
5,98	7,53	9,72

Optimization of the result

MCMC – Optimization of the result

- More steps -> Works until we land in local Minima
- As seen before: Multiple tries of MCMC
- Start in local Minima by reusing MCMC result
 - Do this a couple of time and keep best result:
 - For our example distance: 10.171 -> 10.046 (50 tries)
- If group of points are clearly separated -> MCMC on subgroup and find cheapest connection between those

Further more complex Problems

Further more complex Problems

- Asymmetric distance Matrix
- Time dependent distance Matrix (e.g. traffic jam)
- m traveling salesmen
 - find best distribution of N cities for the m traveling salesmen and find shortest path over all $(N + m - 2)!$ possibilities

Literature

- [1] “Double-ended nearest and loneliest neighbour – a nearest neighbour heuristic variation for the travelling salesman problem”, Fernando G. S. L. Pimentel
https://www.researchgate.net/publication/258241208_Double-ended_nearest_and_loneliest_neighbour-a_nearest_neighbour_heuristic_variation_for_the_travelling_salesman_problem
- [2] „TRAVELING SALESMAN PROBLEM -Insertion Algorithms“
https://www2.isye.gatech.edu/~mgoetsch/cali/VEHICLE/TSP/TSP009_.HTM
- [3] “Optimization of the time-dependent traveling salesman problem with Monte-Carlo methods”, Bentner et al., Phys.Rev. E, 64, 036701

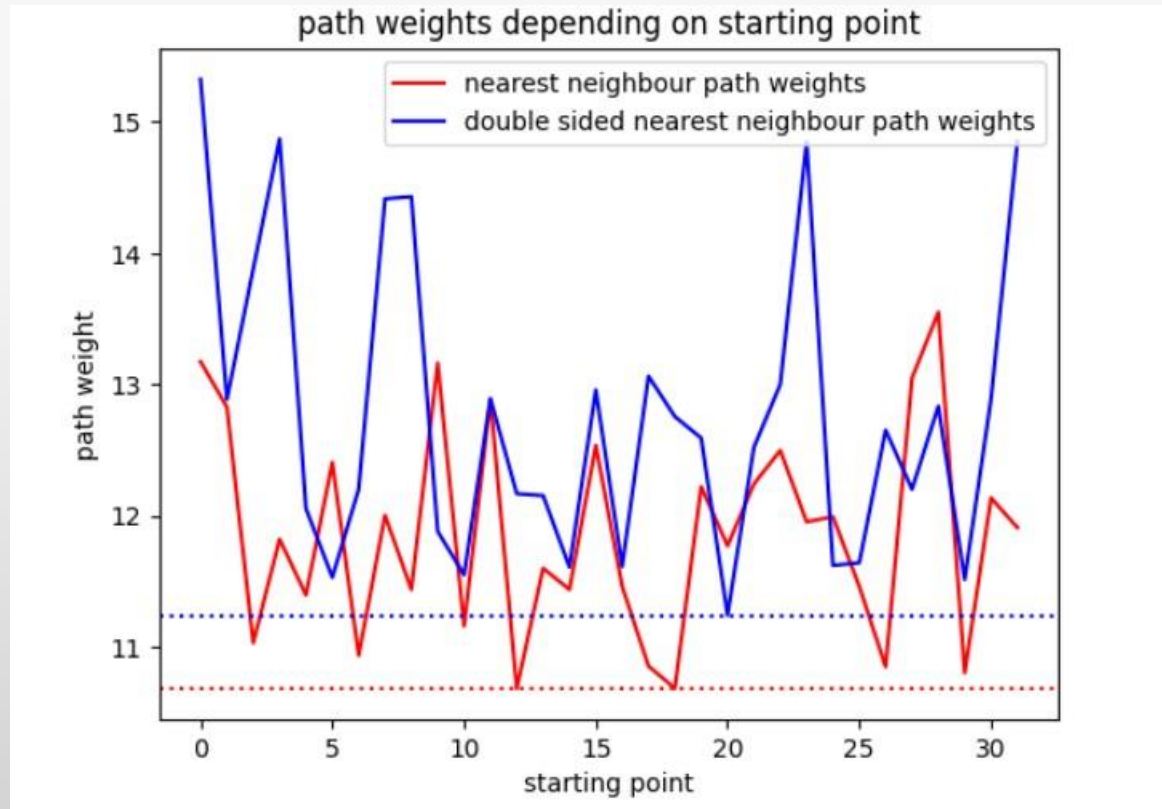
Source for images

- (1) <https://www.tue.nl/en/news-and-events/news-overview/19-12-2023-improving-optimization-efficiency-by-modifying-algorithms-in-tsp-variants>

Back-up

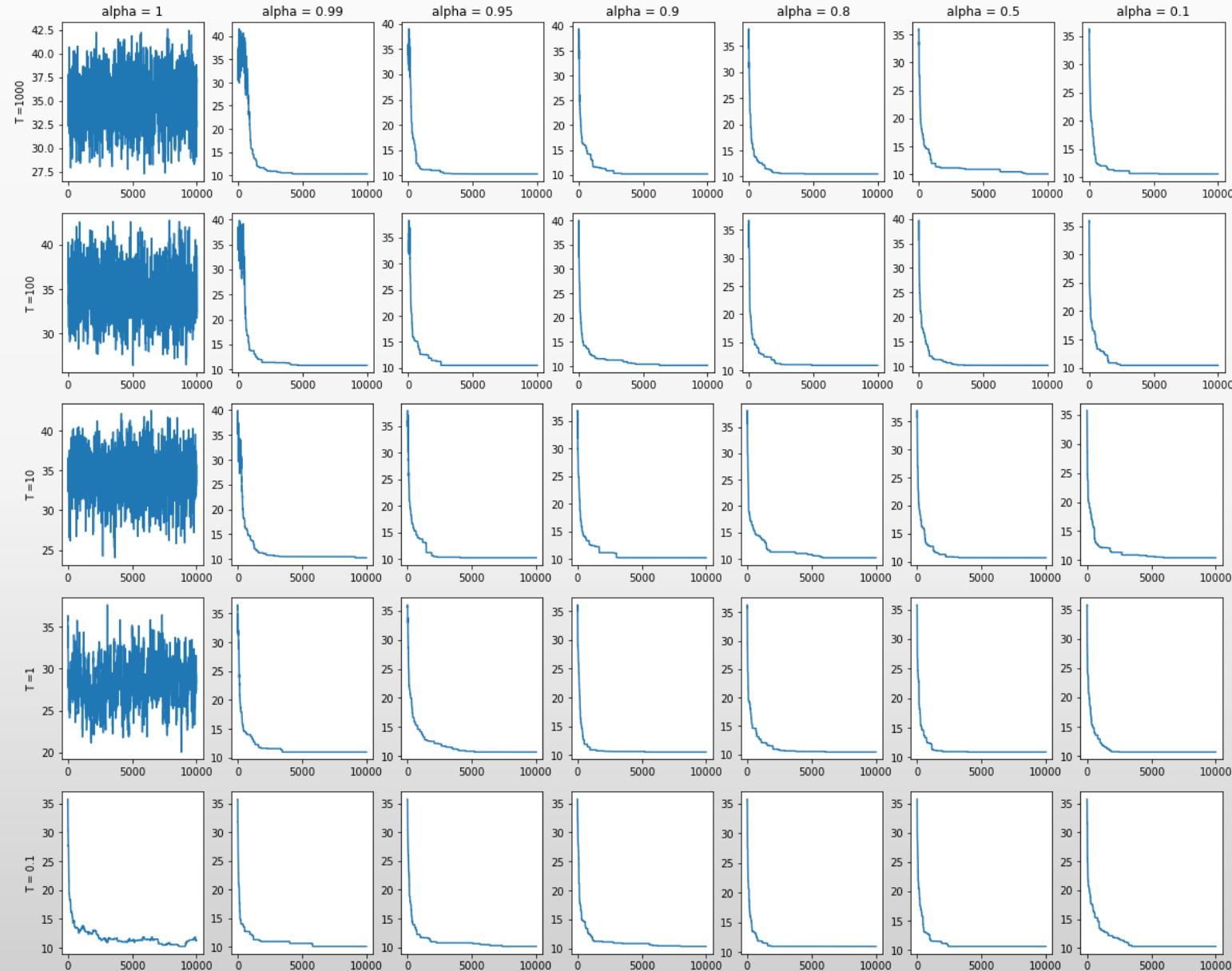
Comparison Nearest with double sided

- Does startposition matter? Yes

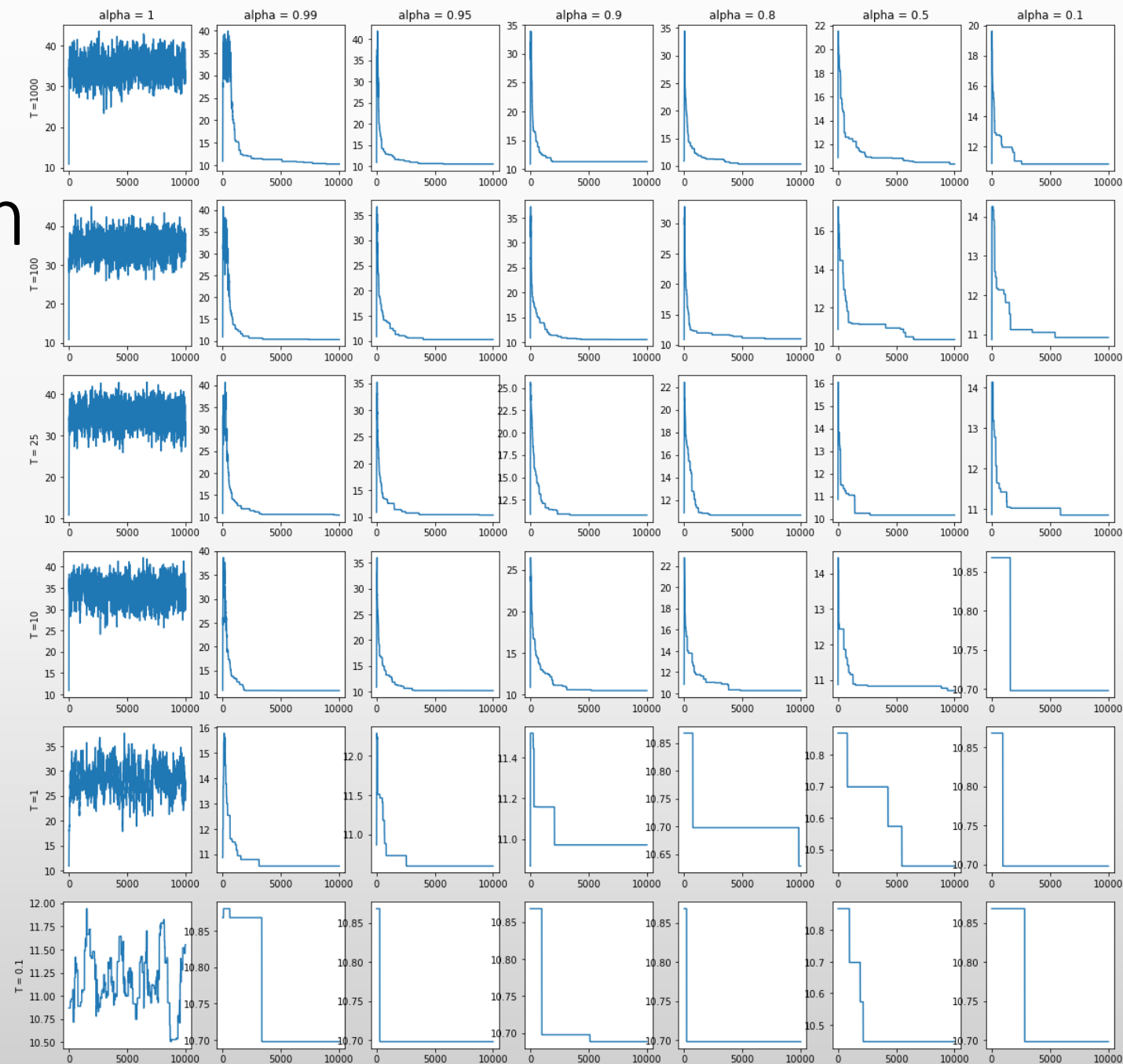


Different T and α 0,1,... start

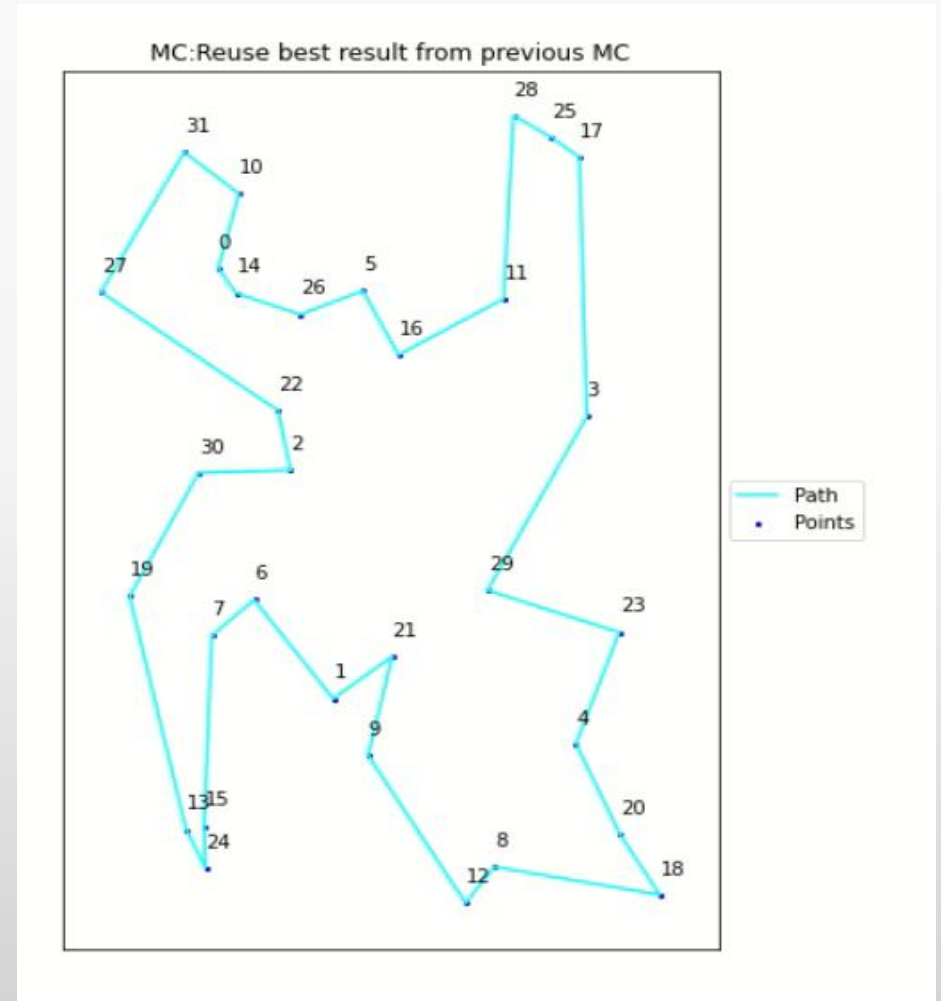
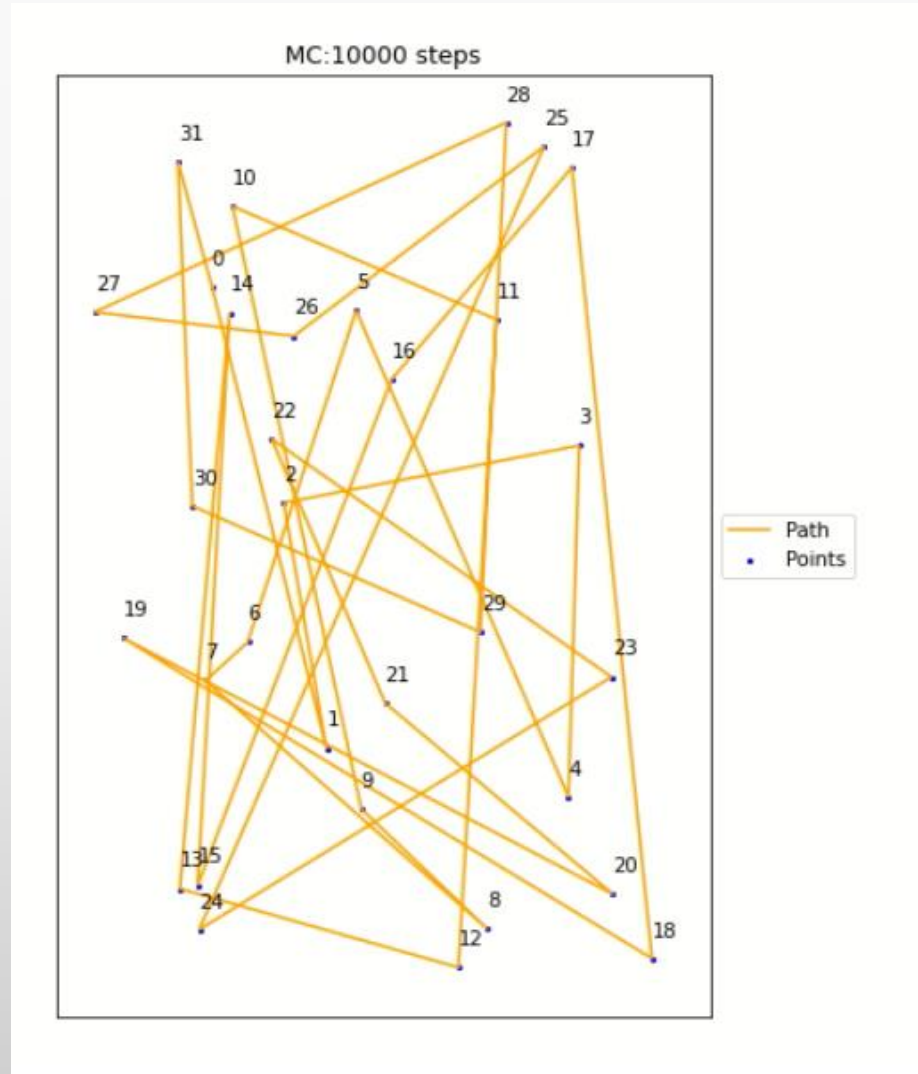
- Result for distance of different α and T with 10000 steps MCMC starting with 0,1,...,32
- We have $\exp\left(\frac{D-D^*}{T}\right)$ with $\max(D - D^*) \approx 2,5$



Different T and α start in local Minimum

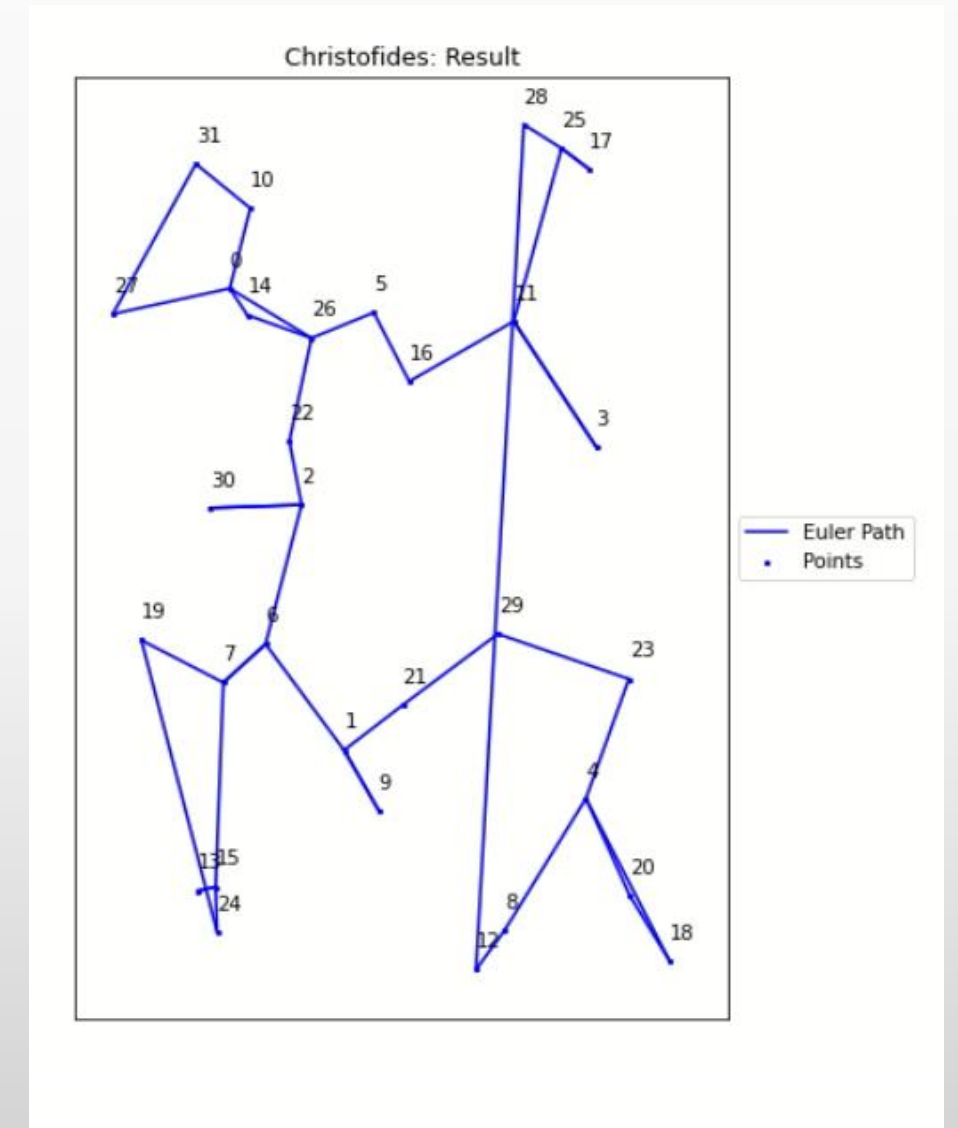


Time comparison local Minima/random start

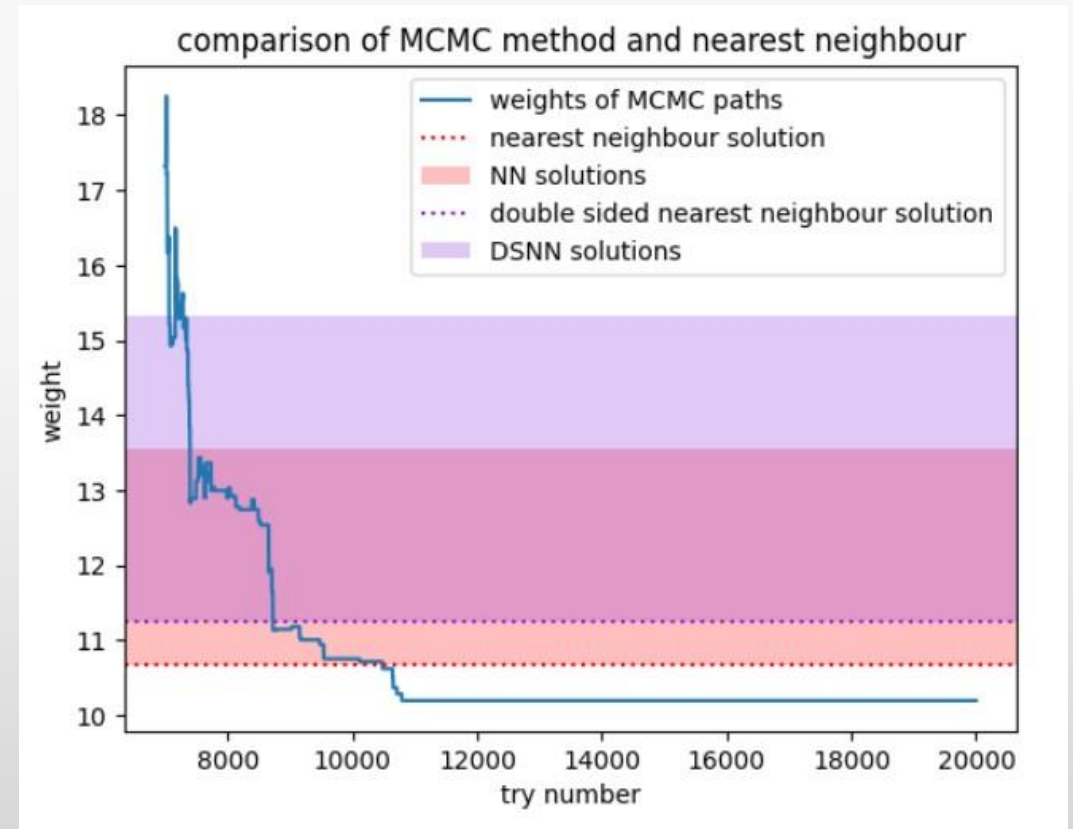
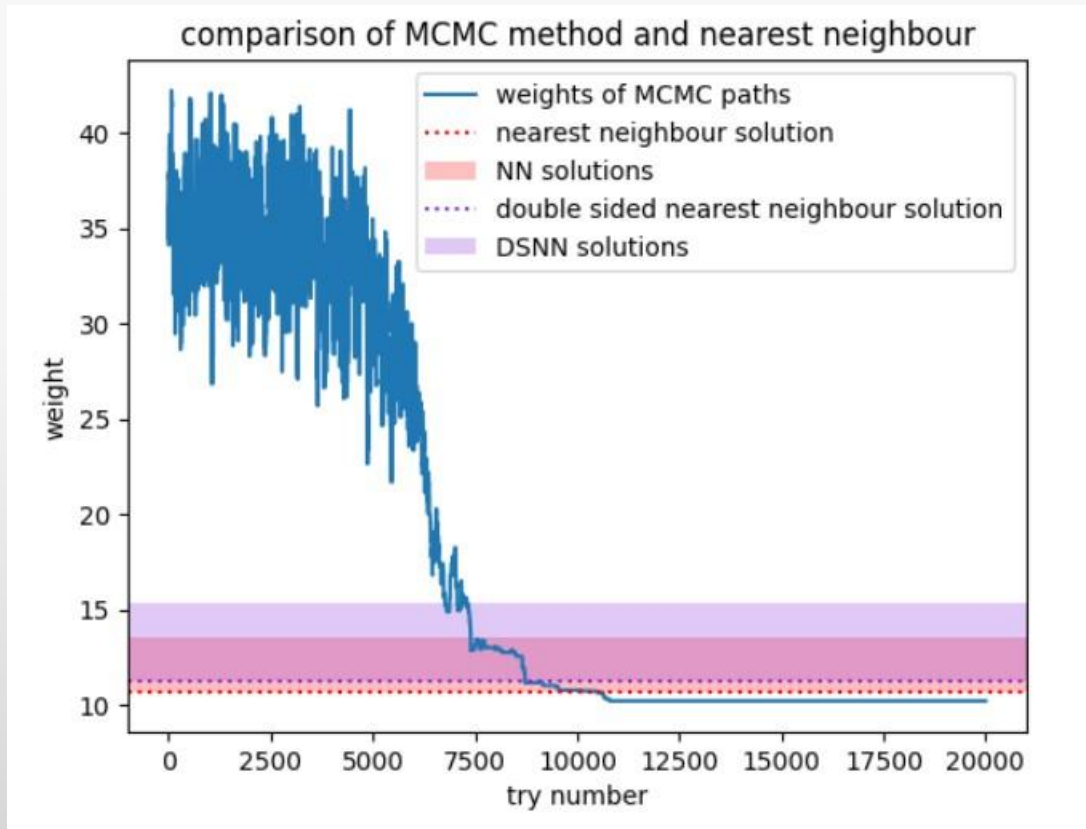


Christofides

- Create a minimal spanning tree T
- Find vertex with odd number of edges
- Find a perfect matching M between those vertex
- Create an eulerian circuit in $T \cup M$
- Create Hamiltonian circuit by skipping repeated vertices starting in a random point
- Minimize over all starting points



Comparison – Distances



$T=100$, $\alpha = 0,99$

Code- Greedy


```

def Nearest_Neighbor(points,distance,start):
    """
    Calculate way with minial distance through Nearest_Neighbor algorithm with given start point
    Input: points - points of interests - integer array from 0-#cities-1
           distances - 2d-matrix containing the distances
           start- startpoint - integer
    Output: way with minimal_distances- by Nearest_Neighbor algorithm

    """
    way = np.array([start]) # start with startpoint

    while len(way) != len(points):
        min_dist = np.max(distance)+1

        for i in range(len(points)):

            if (distance[way[-1],i] < min_dist) and (i not in way):
                # minimal distance and not yet visited city
                min_dist = distance[way[-1],i]
                new_point = i
            way = np.append(way,new_point)

    way = np.append(way,start) # end with startpoint
    return way

```



```

def Double_side_Nearest_Neighbor(points,distance,start):
    """
    Calculate way with minial distance through Double_side_Nearest_Neighbor algorithm with given start point:
    nearest neighbour from left and right.
    Input: points - points of interests - integer array from 0-#cities-1
           distances - 2d-matrix containing the distances
           start- startpoint - integer
    Output: way with minimal_distances- by Double_side_Nearest_Neighbor algorithm

    """
    way_right = np.array([start]) # start with startpoint
    way_left = np.array([start])
    used = np.array([start]) # used
    while (len(way_right)+len(way_left)-1) < len(points):
        min_dist = np.max(distance)+1

        for i in range(len(points)):

            if (distance[way_right[-1],i] < min_dist) and (i not in used):
                # minimal distance and not yet visited city
                min_dist = distance[way_right[-1],i]
                new_point = i
            way_right = np.append(way_right,new_point)
            used = np.append(used,new_point)
            min_dist = np.max(distance)+1
            if len(used) != len(points):
                for i in range(len(points)):
                    if (distance[way_left[0],i] < min_dist) and (i not in used):
                        # minimal distance and not yet visited city
                        min_dist = distance[way_left[0],i]
                        new_point = i
                    way_left = np.append(new_point,way_left)
                    used = np.append(used,new_point)
    way = np.append(way_right,way_left) # end with startpoint
    return way

```

```

def nearest_Insertion(distance):
    """
    Implementation of nearest Insertion method: inserting nodes to cycle, so that length increase is minimal
    Input: distance 2d-matrix
    Output: samllest path regarding nearest Insertion method

    """

    N = len(distance)
    points = np.arange(N)
    x = np.random.choice(points, size=1, replace=False)
    minn = np.max(distance)
    for i in range(len(points)):
        if i!=x:
            if distance[i,x]<minn:
                minn = distance[i,x]
                y = i
    cycle = np.append(x,y)
    while len(cycle) <N:
        d = np.max(distance)
        for c in range(len(cycle)-1):
            pos1 = cycle[c]
            pos2 = cycle[c+1]
            for j in range(N):
                if j not in cycle:
                    if distance[pos1,j]+distance[j,pos2]-distance[pos1,pos2] <d:
                        d = distance[pos1,j]+distance[j,pos2]-distance[pos1,pos2]
                        k = j
                        rep_c = c+1
            cycle = np.insert(cycle,rep_c,k)

    cycle = np.append(cycle,cycle[0])
    return cycle

```

Code- MCMC

```

def twoPointDist(x, y, dist_matrix):
    """ input:  two points x and y - (int)
           matrix giving the distances between points - (2d array)
       return: distance (int) between point x and point y (direction x to y)
    """
    return(dist_matrix[x, y])

def totalTravelDist(points, dist_matrix, loop=True):
    """ input:  points - array of points that gives the path - array
           dist_matrix - matrix containing the distances between all points (in both directions) - 2d array
           loop - info if the path needs to be a loop (start position = end position) - boolean
       output: dist_sum - total weight of the given path - int
    """
    N = len(points)
    if loop:
        dist_sum = twoPointDist(points[N-1], points[0], dist_matrix)
        for i in range(N-1):
            dist_sum += twoPointDist(points[i], points[i+1], dist_matrix)

    else:
        dist_sum = 0
        for i in range(N-1):
            dist_sum += twoPointDist(points[i], points[i+1], dist_matrix)
    return(dist_sum)

```



```

def travelWeightTotal_changingDist(points, distances, periodicity, loop = True):
    """ input:  points - array of points representing the path - array
              distances - matrices containing the distances between all points (in both directions) at different 'times' (3rd dim = time
dim) - 3d array
              periodicity - periodicity of the time matrices in time - int
              loop - info if the path needs to be a loop (start position = end position) - boolean
    return: dist_sum - total weight of the given path - int
    """
    N = len(points)
    dist_sum = 0
    for i in range(N-1):
        dist_sum += twoPointDist(points[i], points[i+1], distances[round(dist_sum) % periodicity, :, :])

    if loop:
        dist_sum += twoPointDist(points[N-1], points[0], distances[round(dist_sum) % periodicity, :, :])
    return(dist_sum)

def exchangeTwoPartorder(x1, x2, y1, y2, arr):
    """ input:  x1, x2, y1, y2 - positions in array at which the array is to be cut in subarrays (it needs to be x1<x2<y1<y2) - int
              array - array which is to be reordered - array
    return: reordered array (exchange the position [0,...,x1,...,x2,...,y1,...,y2,...] to [0,...,y1,...,y2,...,x1,...,x2,...]) - array
    """
    return(np.concatenate((arr[0:x1],arr[y1:y2+1], arr[x2+1:y1], arr[x1:x2+1], arr[y2+1:])))

def Lin_3_Opt (path):
    """ input:  path - array that is to be reordered - array
    return: reordered path (exchanges two random successive parts of the tour without altering their directions) - array
    """
    ordered_cuts = np.array([0,0,0,0])
    while ordered_cuts[1]== ordered_cuts[2]:

        cuts = np.random.randint(len(path), size=4) ##take for random positions in path-array and put them in array
        ordered_cuts = np.sort(cuts)                #sort the array from smallest to largest

    return(exchangeTwoPartorder(ordered_cuts[0],ordered_cuts[1],ordered_cuts[2],ordered_cuts[3],path))

```

```

def Lin_2_Opt(path):
    """ input: path - array that is to be reordered - array
        return: reordered path (Lin-2-Opt move simply reverses the direction of a part of the tour ) - array
    """
    positions = np.arange(len(path))

    cut = np.random.randint(len(path),size =2) ##take random postion in path-array
    order_cut = np.sort(cut)
    change = np.concatenate((positions[0:order_cut[0]], positions[order_cut[0]:order_cut[1]+1][::-1], positions[order_cut[1]+1:]))
    ##make the change with the already ordered positions
    return(path[change])

def Opt_method(x,array):
    """
    Chooses method of path change

    input: x - = 0,1 or 2: 0: Only Lin-2, 1: Only Lin-3, 2: randomize between both
           array - array to be alterad
    output: new array with either Lin-2 or Lin-3 used upon
    """
    if x == 0:
        return Lin_2_Opt(array)
    elif x == 1:
        return Lin_3_Opt(array)
    else:
        method = np.random.choice(2) # choose if you use Lin
        if method ==1:
            return Lin_2_Opt(array)
        else:
            return Lin_3_Opt(array)

```



```

def traveling_MC_constDist(points, dist_matrix, tries, starting_temp = 1, alpha = 0.999, method = 2):
    """ input: points - points that should be visited by each path (in some order) - array
        dist_matrix - matrix containing the 'distances' between each point - 2darray
        tries - iterations of the Monte Carlos - Markov chain - int
        starting_temp - scalar in the exponent of the markov chain acception process - int
        alpha - scalar refigulating the decrease of the temperature after each loop iteration in the MC
        method - see Opt_method- cuurently random between Lin-2 & Lin-3
    return: points - points that should be visited by each path (in some order) - array
        pathWeights - 'distances' (better weights) of each path - array of int
        temps - temperatures after each MC step - array of int
    """
    points = np.append(points, points[0])
    pathWeights = np.zeros(tries+1)
    temps = np.zeros(tries+1)
    pathWeights[0] = totalTravelDist(points, dist_matrix)
    temps[0] = starting_temp
    cost = pathWeights[0]

    for i in range(tries):
        points = np.delete(points, -1)
        new_sequence = Opt_method(method, points)
        new_sequence = np.append(new_sequence, new_sequence[0])
        new_cost = totalTravelDist(new_sequence, dist_matrix)
        if np.random.uniform(low=0, high=1) < np.exp((pathWeights[i]-new_cost)/temps[i]):
            points = new_sequence
            cost = new_cost
        else:
            points = np.append(points, points[0])
            pathWeights[i+1] = cost
            temps[i+1] = alpha*temps[i]
    return(points, pathWeights, temps)

```